

摘要：

桥水首次公开了内部AI研究助手Pat的完整设计思路，他们没有把AI做得更聪明，而是把AI做得更可靠。

世界上最大的对冲基金桥水，最近第一次公开了内部 AI 研究助手 Pat 的完整设计思路。

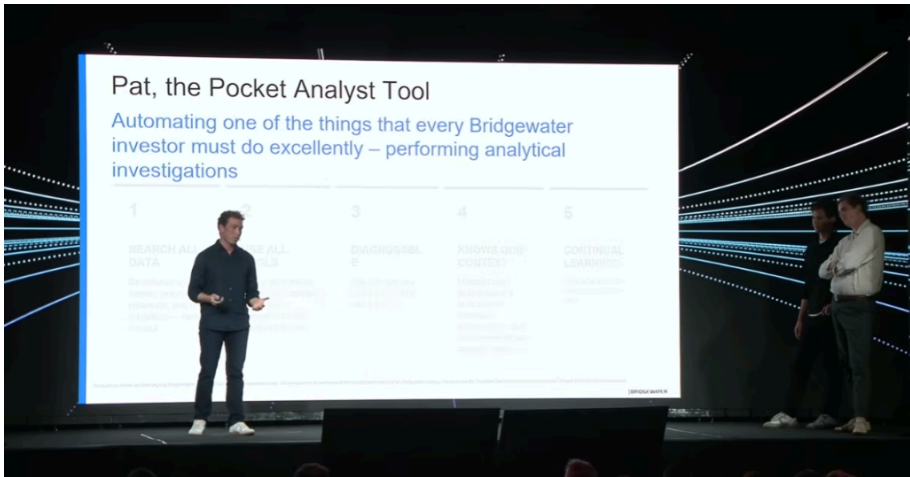
Pat 已经被几百名投资人每天使用。给它一个真实的研究问题——例如“当前中东冲突会不会重演历史上的石油供应冲击”——它会自己找数据、写分析代码、生成交互式研究报告。过去需要研究员几天甚至几周完成的工作，现在几分钟就能开始产出结果。

但桥水这次公开最值得企业学习的，并不是 AI 能做研究，而是另一件事：**他们没有把 AI 做得更聪明，而是把 AI 做得更可靠。**

为了让 AI 可以参与几十亿美元资金的研究决策，桥水没有依赖更强的大模型，而是围绕 AI 建了一整套“确定性”机制：任务先规划再执行、代码强制校验、结果可重复验证、任何一个投资人的经验都可以自动沉淀成整个组织的新能力。

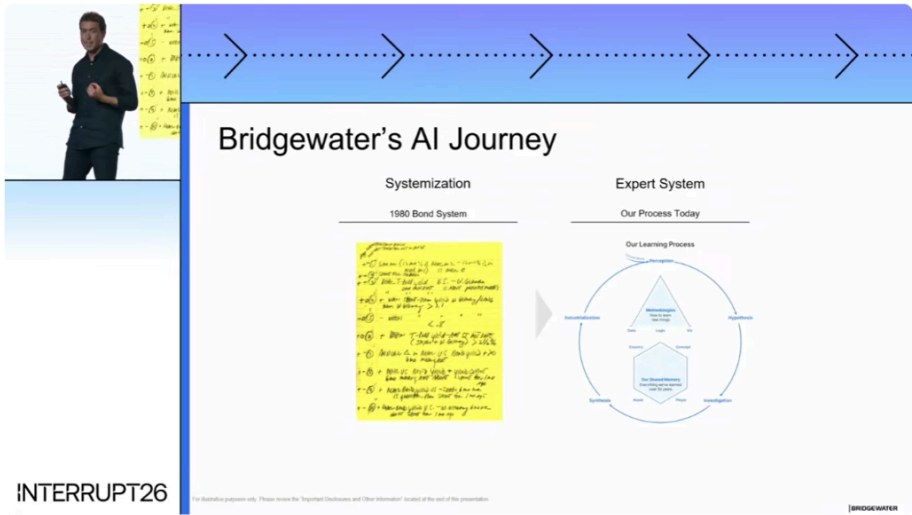
而Pat 背后那条通往 AI 的路，桥水在 AI 还没出现的时候就铺好了。

桥水应用 AI 团队负责人 Brendan McManus、投资负责人 Michael Rand 和技术负责人 Santi Wait在 最近的一场公开技术演讲中，首次把 Pat 的完整设计思路搬到了台前。



攒数据——这件事桥水做了将近50年

Burren 的演讲从一张 1980 年的黄色法律便签纸开始。



那是桥水的第一套债券交易系统。创始人 Ray Dalio 定了一条规则：每做一次交易，把为什么这样做的因果逻辑一条一条写清楚。写下来了，别人才能看、才能挑毛病、才能帮你改进。每次学到新东西，划掉一条旧规则，写一条新的。

这件事桥水做了将近 50 年。每一次交易决策背后的逻辑、每一个市场判断的方法论、每一次翻车的教训——全部被编码成了一套既人可读、又机器可读的专家系统。Burren 的原话是：“我们不需要为了上 AI 临时回去补写知识库。这些数据早就在那里了。”

这个差距，可能是所有企业用 AI 最根本的分水岭。不是谁的模型更好，是谁的数据在 AI 来之前就已经被整理好了。

桥水的 AI 战略分两条线。

第一条线是作为投资者理解 AI 本身——研究 AI 的供需结构、建设周期、对宏观和市场的影响，就像他们过去研究疫情和石油供给冲击一样。第二条线是作为实践者用 AI 改造自己的研究流程——终极目标是打造一个“人工投资者”，把人类投资人每天做的事全部复制一遍。Pat 负责的是其中一个环节：调查分析。

桥水内部是怎么把这件事跑起来的

在讲技术之前，Burren 先讲了一个容易被忽略的维度：组织。

他们没把这个项目交给 IT 部门。他们搭了一个“内部创业团队”——投资者、工程师、科学家跨职能坐在一起，小团队敏捷迭代，同时调用桥水的大机构资源。

投资者的角色是定义目标——研究什么才有价值。工程师的角色是架构实现——系统怎么搭才能稳定。科学家的角色是严格评估——怎么确定 Pat 的分析质量真的在变好，而不仅仅是“看起来不错”。

几百个投资人每天都在用，持续提供真实的反馈信号。这个飞轮一旦转起来，产品的进化方向就不是靠产品经理猜的，是靠几百个专家用户的行为推的。

两个Agent，各管一摊

Pat 的系统架构不是一个万能大模型在运转。是两个独立的智能体分了工。



一个叫聊天智能体（Chat Agent）。它只跟投资人对话，只谈投资内容——“这个数据够不够”“要不要再看一个角度”“当前版本下哪些因素是主导”。投资人不是程序员，不应该被迫关心代码。所以这个聊天智能体从头到尾不暴露任何技术细节——从对话里，用户看不出后面有代码在跑。

另一个叫编码智能体（Coding Agent）。它不跟用户说话。它的工作只有一件：把聊天智能体产出的计划，“编译”成 Python 和 Pandas 代码。

Santi 说，分开之后产生了三个意外的收益。

第一，投资人体验更自然——用起来就像一个懂行的研究助手，不是在用编程工具。第二，两个智能体各自专精、上下文互不污染——聊投资的不用管代码，写代码的不用理解投资的语义。第三，可以把对话流程产品化成一套可靠的固定工作流，而不是一堆散装知识。

编译器式的确定性：同一个问题问两次，95% 的情况下代码一模一样

这是 Pat 整个架构里最硬核的部分。

Santi 说，在这个对正确性要求极高的场景——几十亿美元的仓位——不能靠 vibe coding。Vibe coding 的问题是你每次跑出来的结果都不一样，这次对了不代表下次也对。对一个对冲基金来说，这个不确定性承担不起。

于是他把 agentic coding 当作编译器问题来解。

第一步，计划不是待办清单，是“自然语言版的 Python 项目”。计划里把这次分析要产出的所有数据表（dataframe）、每张表的字段和结构、表与表之间的依赖关系，全部提前写好。每个子任务像一个 Python 函数——有明确的输入依赖和输出规范。目标是让同一个计划被不同的大模型实例执行时，生成的代码在语义上完全等价、输出完全一致。

第二步，代码是并行生成的。因为有清晰的依赖关系图，系统先做静态分析，得到一张 DAG（有向无环图），然后按层级并行生成代码。一个分析里有 3 张数据表和 30 张数据表，代码生成的时间几乎一样。

第三步，校验不是“智能体自己决定要不要检查”。Santi 特别强调，他们的 harness 是常规的 Python 代码——校验是强制的、写在架构里的。“一个智能体可能会‘忘记’做检查，但一条固定的代码管线不会。”在这个约束下，他们在测试套件上拿到的结果是：同一个计划两次生成的代码，95% 的情况下完全一致。

第四步，执行时不是让大模型跑代码。走的是传统静态分析管线，自动注入缓存标注——哪些数据已经加载过了、哪些计算已经跑过了，第二次不用重来。结果是，如果用户只是改了一个图表的标题，传统工具可能重跑整个分析流程，Pat 几乎瞬时就能出结果。越改越快。

数据检索多了一步——像人类一样审视

在正式分析前，Pat 需要先搜数据。

桥水有两个数据库。一个是非结构化的——几百万篇券商研报、财报电话会纪要、内部邮件和备忘录，近实时更新，每天新增几千份。另一个是结构化的——几十亿条时间序列数据，既有外部数据（油价、股指），也有内部建模的概念（比如“我们预测 12 个月后的通胀率”）。

搜索本身用了 RAG 和重排序这些常规手段。但 Santi 的团队发现，加一步“人类式审视”之后，命中准确率从大约 50% 跳到了接近 90%。

“人类式审视”的意思是，搜到一个数据序列后，不只靠名字匹配——要像人类研究员一样去检查：它的频率对吗？币种对吗？数值和你已有的判断一致吗？这一步把模糊的关键词搜索变成了带判断的数据校验。

自我进化机制

Pat 有两套自我进化的机制。

隐式的：后台智能体会自动扫描已完成的对话，找出失败的模式，生成验证基准，确认问题可以复现，然后自动迭代 Pat 的知识库和系统指令，直到这个基准通过。

显式的更直接。用户在分析过程中如果觉得 Pat 应该做得更好——比如应该多出一组图表、应该提前想到一个角度——可以点一下“教它”按钮。

系统会自动回溯整段对话，识别到底是行为错误、上下文缺失、还是用户的引导本来可以被预判。然后自动生成一个“应当失败”的验证用例、确认问题可复现、自动修改知识库直到这个用例通过、再跑一遍所有已有验证确保没引入新问题。最后一条 Pull Request 发到 Slack 里，人审后合入。

下一次任何人来问类似问题，默认拿到的就是改进过的版本。

以下为演讲文稿，由AI辅助翻译：

桥水基金应用AI团队负责人Brendan McManus

大家好，我叫 Brendan McManus，是桥水基金（Bridgewater Associates）应用 AI 团队的负责人。桥水是一家系统化宏观对冲基金。我在桥水已经工作了将近十年，最开始是一名软件工程师，之后成为了一名系统化投资者和研究员。过去几年里，我主要专注于弥合投资与技术之间的鸿沟。

今天，我和我的同事 Michael Rand 以及 Santi Wait 一起来到这里。两位分别是该项目的投资负责人和技术负责人。我们将向大家介绍一款由我们内部开发的优秀工具，叫作 Pat——“口袋分析师”（Pocket Analyst）。

到本次演讲结束时，大家将会看到：我们如何构建出一名 AI 分析师，它能够在几分钟内完成专家通常需要数小时才能完成的研究工作。

这款工具目前已在公司内部部署，供数百名投资人员使用；并且，它能从每一次交互中持续学习。此外，我们还会向大家展示，我们究竟是如何设计并构建这样一个系统的。

在正式展示我们构建的产品之前，我想先简要介绍一下桥水看待 AI 的方式。

桥水已经花了数十年——准确说是 50 年——思考如何将市场和经济规律编码为能够不断复利增长的系统。而这一切，真正始于你们屏幕上所看到的内容：我们在 1980 年写在黄色便笺纸上的债券系统。

核心理念非常简单：每当你想做一笔交易时，都要准确写下你认为这笔交易成立的规则；写下其中确切的因果逻辑。因为一旦这么做，其他投资者就能查看你写下的内容，帮助你找出哪里出错



了，并协助你改进整个过程。

每当你学到新东西，就划掉一条旧规则，再写下一条新规则。这样就形成了一个非常强大的学习过程，而这也正是过去 50 年来桥水一切工作的基础。

几十年来，我们大幅积累和强化了这一过程。我们将关于所做交易、交易方式及其原因的每一项经验、每一种方法论、每一条规则，全部编码进了一个既可供机器读取、也可供人类理解的专家系统之中。

如今，我们拥有极其丰富的数据资产。而正是这些数据，让我们能够很好地迎接 AI 时代。我们不需要回过头去专门为智能体重新记录所有内容——这些内容本来就已经存在，供我们调用。

在进一步介绍这款工具之前，我想谈谈桥水在更广泛层面上如何应用和理解 AI。我们从两个角度推进这件事。

第一个角度是：作为投资者，我们必须深入理解每一个正在塑造全球市场和经济的重要趋势和动力。

正如我们必须理解新冠疫情，或者近期石油供应冲击一样，我们同样必须理解 AI。AI 领域的供需失衡呈现出怎样的形态？推动基础设施建设和扩张的因素是什么？这些因素最终又会如何影响市场？

对于我们这些投资者来说，理解这些问题只是基本门槛。

第二个角度是作为实践者来使用 AI，这也是今天我们主要讨论的内容。

作为实践者，我们正在把 AI 应用于研究流程的各个环节，最终目标是构建一名“人工投资者”——它能够执行我们的人类投资者每天所做的全部活动。

那么，人类投资者实际都在做什么？

我们把它看作一个“研究循环”。投资者持续感知外部世界中正在发生的事情，提出关于什么是真实的、自己可能遗漏了什么问题；然后开展分析调查来尝试回答这些问题；接着整合研究发现；最终，把所有学到的内容重新纳入我们持续积累的知识体系和专家系统之中。

这里最后一步尤为关键。

通过这一流程获得的所有知识，都会被存入一个共享记忆体系中，供人类继续调用。

因此，你可以想象：如果要构建一名人工投资者，它就必须能够完成研究流程中的每一个不同步骤。你可以设想建立一系列离散的子智能体，每一个专注于研究循环的某一个环节。

这正是我们使用 AI 的方式。我们为人类投资者必须完成的各种任务分别构建专门的子智能体，并让它们调用我们过去 50 年积累起来的同一套知识和理解。

不过，今天我们只会讨论其中一个智能体：它专注于研究流程中的“调查”环节，即那些需要人类分析师花费数天乃至数周才能完成的深度分析工作。

我们把这项工具命名为 Pat，即“口袋分析师工具”(Pocket Analyst Tool)。

这里也要先明确一下预期：Pat 并不直接涉及我们如何交易。它真正的用途是进行深度探索式研究，让投资者能够研究那些过去由于时间和精力限制、根本无暇研究的问题。



那么，我们具体构建了什么？

我们构建了这款名为 Pat 的口袋分析师工具。从第一天起，它的产品规格就很简单：必须让 Pat 能够完成我们的人类投资者在调查和分析工作中所做的一切。

而这首先意味着数据能力。

Pat 必须能搜索并阅读我们内部的所有不同类型数据，包括：

结构化时间序列数据，例如跨越数十年的股票价格；

非结构化数据，例如我们订阅的经纪商/交易商研究报告；

我们内部撰写的研究备忘录。

Pat 必须能够搜索并阅读所有这些内容。

Pat 还必须能使用人类分析师可使用的各种工具，包括我们自主开发的可视化工具、诊断工具，以及用于评估指标创意质量的工具。

此外，我认为对于技术听众而言，这一点尤其有意思：Pat 所运行的许多分析，单次就需要人类分析师花费数小时完成。

这意味着分析本身相当复杂。因此，Pat 的分析必须是完全可诊断的——不仅要让人类能够诊断，也要让后台运行的智能体能够读取执行轨迹、理解过程，并确认每一项计算都是正确的。

除此之外，我认为过去 50 年持续记录和积累的知识，在这里开始真正发挥价值：Pat 知道我们的全部上下文。

它可以访问我们的投资流程和框架；它准确知道我们的分析师应当如何开展工作，因为过去 50 年我们一直把这些东西记录下来。

最后，Pat 必须能够学习。

它不仅要为某一位投资者积累学习成果，还必须能够为公司里的每一位投资者积累和复利这些学习成果。

Pat 今天并不是一个原型产品。它实际上已在数月前完成内部部署，目前已有数百名投资人员每天都在使用它。

这带来了一个相当强大的改进飞轮：当投资者使用 Pat 进行真实研究时，后台会持续运行智能体，扫描这些互动过程，识别 Pat 出错的地方，构建经过人工审核的基准测试；随后，这些发现会推动我们修改上下文内容以及为 Pat 构建的运行框架。

因此，Pat 的改进不只服务于一个人，而是服务于所有人。

最后，在介绍产品之前，我想回答一个经常被问到的问题：一家已有 50 年历史的对冲基金，究竟是如何打造出你们马上将看到的这种产品的？

一切首先始于：你必须有能力、也有意愿去重塑自己。



构建 Pat 的团队，本质上是一个在公司内部孵化出来的应用 AI 创业团队。我们既能够非常灵活、快速地行动，也能够调用整个公司的资源。

此外，我们建立了多种角色共同协作的团队：投资者、技术人员和科学家并肩工作、共同构建产品。

我认为，如果你要为专家用户构建这样的产品，这一点至关重要。

投资者带来业务背景和领域专业知识；

技术人员带来系统架构能力；

科学家带来严谨性。

如果要为专家用户构建 AI 系统，正是这种多角色团队不可或缺。

说到专家用户，我们内部有数百人正在使用各类 AI 工具，不只是你们马上会看到的 Pat。他们每天都在提供信号，帮助我们判断这些工具应如何持续演化和改进。

最后，我们还有一个非常出色、能够持续复利的生态系统可以接入：50 年积累的数据、工具和方法论。这些资源不仅供人类分析师使用，也同样供智能体使用，服务于我们构建“全人工投资者”的长期旅程——让它最终能够完成今天人类所能完成的一切工作。

接下来，我把时间交给我们的投资负责人 Michael Rand。他将为大家演示我们构建的产品，并介绍其产品架构。

桥水口袋分析师项目投资负责人Michael Rand

好的，谢谢 Brendan。

我叫 Michael Ryan，是口袋分析师项目的投资负责人。简单介绍一下我自己：我在桥水已经工作了五年，最初是以技术人员身份加入的，但之后大部分时间都担任投资相关岗位。

目前，我最主要关注的问题是：如何将 AI 融入我们的投资流程。

接下来，我想直接演示一下口袋分析师，展示它的能力，以及桥水的投资人员如何使用它。

大家身后的屏幕上可以看到 Pat 的主页，其中有我们今天演示所使用的提示词。

这个提示词的大意是：我们要求 Pat 研究市场对近期中东冲突的反应，并将当前事件与类似的历史事件进行比较。最后，我们要求 Pat 制作一系列可视化图表，以突出当前情况与过去石油供应冲击之间的相似性和差异性。

这是桥水过去几个月一直在研究的真实问题，而 Pat 一直被投资者用于加速我们的研究过程。

不过，在提交这个请求之前，我想先花一点时间讲讲我们在设计 Pat 运行框架时遇到的一个有意思的安全问题。

我们的出发点是：如果 Pat 要真正发挥作用，它就必须能够访问投资者在开展研究时有权访问的全部信息。

但问题在于，在桥水，不同投资者能够访问的信息不同。



举例来说，某位投资者可能有权限查看我们目前在所有市场上的持仓情况。对于这个人而言，他所使用的 Pat 也必须能够访问这类信息。

但与此同时，也有一些分析师并不接触这些信息。因而，同样至关重要的是：我们绝不能意外将这些受保护的知识产权或敏感信息泄露给这些分析师。

所以，与 Claude Code 这类所有人都使用相同系统提示词和相同工具的运行框架不同，桥水中的每个人都拥有一个独特版本的 Pat，它根据该用户可以和不可以看到的信息进行定制。

从实践角度说，这种差异本质上取决于每个人的 Pat 所拥有的上下文和工具。

现在，我们提交这个提示词。分析开始后，Pat 做的第一件事就是搜索网络，以及我们内部的非结构化数据仓库，从而更好地理解当今世界正在发生什么，并将其置于历史背景中进行解读。

对于现代聊天应用来说，网络搜索本身只是基础能力。

但这里真正的差异化优势在于：Pat 能够搜索的非结构化内容范围非常广泛，这来自我们订阅的各种信息资源。

我们有一个包含数百万份文档的数据库，覆盖世界各地的信息，其中包括：

经纪商研究报告；

财报电话会议记录；

内部邮件；

以及更多内容。

这个数据库近乎实时地更新，每天会新增数千份内容。

这也呼应了我刚才所说的观点：如果 Pat 要真正提供杠杆效应，它就必须能够访问其用户所能访问的一切信息，从而尽可能模拟用户实际的工作方式。

当收集完这些上下文后，Pat 接下来会搜索我们的时间序列数据库，寻找完成分析所需的数据。

这个数据库包含数千万条时间序列，是我们过去 50 年里持续在内部建模和积累的成果。

其中既包括来自外部世界的的数据，例如石油价格；也包括我们内部推导出的概念，例如我们预测未来 12 个月的通胀水平。

搜索智能体会使用一些传统的搜索技术，比如 RAG（检索增强生成）、重排序等。

但我们发现，一个真正带来巨大改进的因素，是在其中加入类似人类研究员的“检查”环节。

我的意思是，当人类研究员寻找数据时，他们并不只是盯着时间序列的名称。他们还会查看很多东西，例如：

该序列的频率；

该序列所使用的货币；



最重要的是，该序列的取值是否与他们原有的判断和先验认知相一致。

将这类推理机制嵌入我们的搜索智能体，使我们的准确率从大约 50% 提升到了 90%。

在 Pat 获得了所需的上下文和数据之后，它会向用户提出澄清问题，并可能提出在分析过程中值得进一步探索的其他角度。

在 Pat 的开发过程中，我们逐渐形成了一个观点：计划本身就是分析。

如果我们能够制定出高质量、足够详细的计划，我们就有信心能够持续地、智能地执行这一计划，并产出我们希望得到的结果。

Santi 稍后会更深入地介绍这一点。

如今，智能体会提问这件事在聊天机器人中已经很常见了。但我们真正关注的是这些问题的实质内容。

我们投入了大量时间和精力，开发上下文和基准测试，以塑造这种能力。我们教会 Pat 什么是好的研究问题，什么是不好的研究问题。

这种来回互动能够帮助通常在规划环节投入不足的人类用户，逐步形成我们认为高质量的研究计划。

当所有模糊点都得到解决后，Pat 就进入规划阶段。

在规划阶段，它主要做三件事：

明确分析过程中将要生成哪些数据框（DataFrame）；

确定每一个数据框的模式（schema）；

最重要的是，明确这些数据框之间如何相互连接。

从时间角度来看，这个规划阶段在整个分析过程中成本相对较高。

但这是我们有意承担的成本，因为它使我们能够在执行阶段实现更高效的能力。

现在，计划已经确定。计划执行的第一步是生成代码。

由于我们的计划极其详细，因此我们可以利用子智能体，并行地为分析中的每一个数据框生成代码。

之所以能够这样做，是因为每一个子智能体都知道：

它依赖哪些数据框；

那些数据框的模式是什么；

它自己所要生成的数据框应当具有怎样的模式。

因此，无论一项分析只包含 3 个数据框，还是一项较为复杂的分析包含大约 30 个数据框，生成代码所需的时间大致是相同的。



当完成代码生成后，我们会执行这些 Python 函数，同时由一个智能体监督执行过程。

如果它发现运行时错误、无意义的数值等问题，就会介入处理。

在继续讲执行过程之前，我想特别指出：这些分析生成的时间序列输出，会落入与输入数据相同的数据库中。

我认为这很重要，原因有两点。

第一，它表明 Pat 分析的任何输出，与我们多年来由人类上传和生成的任何时间序列，在系统中是没有区别的。

第二，也是更重要的一点，Pat 分析的任何输出都可以作为下一次分析的输入。

这样一来，就形成了一个环境：人类和智能体能够非常轻松地复用彼此的成果，并在彼此成果之上持续积累。

执行完成之后，就像你希望初级分析师在向你汇报前先复核自己的工作一样，我们也希望 Pat 做同样的事情。

因此，在分析的这个阶段，Pat 会检查它计算出的数据和生成的可视化图表，确认数字是否合理、图表是否整洁清晰。

如果它发现看上去有些不对劲的地方，Pat 会后退一步，诊断问题，然后改进分析过程；在确认自己对结果满意之后，才会把结果交给用户。

最终交付物是一份交互式报告。

在这份报告中，图表的外观与桥水投资者平时制作的图表完全一致。Pat 使用的是相同的内部图表库，也利用了我们数十年来在内部持续开发的同一套文本和图表体系。

你们可以看到，用户可以放大或缩小这些图表；还可以把这份交互式报告中的数据发送到我们的内部图表工具中，以便即时进行进一步调整。

在把时间交给 Santi 之前，我想再谈谈：Pat 如何随着使用而不断变得更好。

主要有两种方式。

第一种是 Brendan 之前提到的自主式学习：我们让智能体审查已完成的对话，寻找能够让 Pat 变得更聪明的方式。

第二种，也是我们这里要展示的，是更显式的方式：如果用户认为自己与 Pat 的互动中存在值得学习的内容，那么他可以在分析的上下文中直接启动这个学习流程。

这里，用户只是要求生成一组不同的可视化图表。

请注意，用户并没有说之前的内容有错。他只是希望从另一个视角来看问题，并认为这个视角对于回答当前问题很重要。

如果用户认为 Pat 从一开始就应该生成或建议这组图表，那么他可以点击“教学”(Teach) 按钮。

这会启动一个智能体，审查整个对话，寻找例如以下问题：



行为失误；

上下文缺口；

本可以提前预判并满足的用户引导或偏好。

随后，用户可以对这些内容进行修改，也可以按原样提交。

提交后，首先，后台智能体会创建一个预期会失败的基准测试。这证明我们能够复现之前这种较差的行为。

接着，它会持续迭代我们的上下文知识库或运行框架本身，直到这个基准测试通过。

然后，我们会确认：让这个基准测试通过，并没有导致测试套件中的其他测试失败。

完成后，我们会收到一条 Slack 消息，其中附有一个拉取请求（Pull Request），包含该智能体希望对 Pat 作出的修改。

这样做的效果是：下次当有人带着类似问题来使用 Pat 时，他们从一开始就可以获得改进后的版本。

现在，我将把时间交给 Santi，由他来介绍技术架构。

口袋分析师项目技术负责人Santi Wait：

大家好，感谢 Michael。我是 Santi，是口袋分析师的技术负责人。

我相信在场的很多人都在构建类似的编程智能体产品，就像我们一样。

而构建这类产品确实非常困难。

编程智能体非常反复无常，也难以预测；它们经常会犯错；而当你特别倒霉时，它们甚至可能彻底失控，试图毁掉你的数据，以及诸如此类的问题。

所以，要构建一个人们喜欢使用的好产品，本身已经很难；而要构建一个用户愿意真正嵌入日常工作流的产品，则更难。

在一家对冲基金中，我们试图交易数十亿美元的资金。因此，我们不能让所谓“凭感觉写代码”（vibe coding）成为这些分析背后的基础。

我的背景是编译器理论和编程语言设计。

而编译器具有非常类似的一组要求：必须完全确定性、完全正确且可靠。

例如，在驾驶飞机时，你不能出现“差一错误”（off-by-one error）。

这里的结构也非常相似：编译器接收用户代码，并将其编译为例如 JavaScript 之类的目标代码；而编程智能体接收用户提示词或计划，并将其“编译”为 Python。

我们非常喜欢这种思路。



今天时间不多，因此我们将重点讨论这一点，希望它能成为一种经验，让大家带回自己的工作中。

不过，我们还是从聊天环节开始。

聊天智能体的目标，是与用户形成共同理解：用户希望完成什么任务。

聊天智能体是使用 LangGraph 实现的。

我们主要使用它来支持持久化功能。它原生支持取消和续接功能；以前我们自己管理这些问题，效果要差得多。

聊天智能体可以调用工具。Michael 刚才提到了一些，例如：

时间序列数据搜索；

非结构化数据搜索。

这些工具各自会完成对应的工作。

随后，一旦聊天智能体了解完成分析所需的全部数据，它就会制定一个计划，并调用一个子智能体——也就是我们的编程智能体。

这个编程智能体将生成一份基于 Python Pandas 的分析。

为什么要把两个智能体分开？

在早期，我们就决定：我们的投资者并不是职业程序员，他们关心的是投资。

因此，我们决定让聊天内容纯粹围绕投资主题。

这样做的结果是，我们获得了一个产品：编程只是纯粹的实现细节。

在聊天界面中，你甚至无法感觉到后台有代码在运行。

还有一些意外收获：

上下文不会被代码细节污染；

每一个智能体都能专注于自己的任务，并自然地不断改进；

我们可以对聊天体验进行大量定制。

接下来，我们想讲的是：我们的投资领域上下文质量非常高。

我们教聊天智能体如何像一名桥水投资者那样说话。

这里有许多专业术语需要教给它。因此，用户和智能体之间的交流方式，就像桥水内部的同事之间在交流一样。

在桥水，我们也允许投资者作出贡献。Michael 刚才展示的，就是他们像开发者参与代码库一样，为系统贡献内容。



这其实是一个非常好的观点：大多数情况下，你的用户比你更擅长编写业务上下文。

因此，不要自我意识过强，允许他们直接参与贡献，是成功的一个好办法。

关于聊天智能体，最后一点是：我们不仅仅向它教授上下文。

如果只给它上下文，最终得到的往往是某种模糊的、信息量很高但不太像工作流程的东西。

因此，我们会为智能体提供关于如何处理特定类型分析的逐步指南。

这样一来，它更像一个真正的产品，像是可靠、可依赖的工作流程。

接下来，我要谈的是我个人最喜欢的部分：编程智能体。

这是一张编程智能体的高层架构图。

你们在这里看到的所有东西，实际上都只是 Python 代码。它是用 LangGraph 实现的，但并不存在所谓“智能体式编排”(agentic orchestration)。

很多人在拍照，很好。

我们从最左边开始讲，也就是由聊天智能体生成的分析计划。

分析计划会被拆分为若干任务。

每一个任务大致对应一个 Python 函数，该函数将计算并生成一个数据框。

这里是一个模式示例。

每项任务都会包括：

一个名称；

对需要计算内容的描述；

关于输出数据框的结构性和语义性信息。

我们期望每一个任务都可以通过大语言模型，以确定性的方式被编译成一段代码。

也就是说，两个大语言模型针对同一个任务生成的代码，在运行后应当在语义上等价，输出值也应完全一致。

因此，我们的分析计划并不只是像 Claude Code 中可能出现的待办事项列表。

我们将其视为一个“用自然语言写成的 Python 项目”。

而因为其中包含如此丰富的细节，我们现在就可以进入代码生成阶段，并应用一些更高级的技术。

首先，我们将计划拆分为任务，然后并行进行大语言模型代码生成。



由于计划足够详细，计划末尾的一个可视化任务，实际上已经知道自己需要消费和使用的全部内容——即便用于加载数据的代码生成尚未完成。

当我们把这一方法与 Claude Code 在我们的基准测试套件中进行比较时，可以看到：在相同上下文和相同计划下，我们平均生成代码的速度大约快 4 倍。

此外，我们还具备一种“超扩展”能力：一个包含 20 个任务的计划，与一个只有 3 个任务的计划，所需时间基本相同。

好了，现在我们有了代码，接下来要执行它。

你可能会想，只需要直接、朴素地执行代码就可以了。

但遗憾的是，今天的大语言模型在我们的任务上还不够完美。它们通常无法一次就生成完全正确的结果。

因此，我们的做法是：

我们拿到计划中的任务，也拿到由该任务生成的代码；然后运行这段代码，将执行结果与任务要求进行比较，检查它是否正确。

如果不正确，我们就编辑代码并继续迭代，直到完成为止。

我们对代码所做的第一件事，是进行静态分析，然后构建 DAG（有向无环图），并并行应用验证智能体。

在这里，有两个任务会同时接受验证。

因此，一个包含 5 个任务的计划，会被拆分为 3 层验证；而一个包含大约 20 个任务的计划，可能会形成 4 或 5 层验证。

我希望大家从这一部分幻灯片中真正带走的核心观点是：

我们在架构层面强制保障正确性。

再次强调，没有智能体式编排。这只是普通的 Python 代码。

因此，整个执行框架非常快速，而且智能体不可能“忘记”进行验证——它们被强制要求进行验证。

结果是，当我们运行测试套件时，对于测试套件中的任一计划，在 95% 的情况下，两个不同智能体所生成的代码结果完全一致。

因此，我们的理念是：构建一个具有确定性特征的文本编程智能体。

而因为这个智能体具有很强的可复现性，所以当我们要扩展、逐步爬坡优化以及进行评估时，我们拥有的基础要比那种依赖感觉、或依赖“大模型充当裁判”（LLM-as-a-judge）的评估方式可靠得多。

好的，还有一个话题我想谈一下，那就是我们的执行层。

通常，编程智能体会自行调用和执行它们生成的代码。



也就是说，它们先生成代码，然后通过终端自己调用并运行。

这种方式有一些权衡。

一方面，工具调用会带来较高延迟；另一方面，我相信大家都经历过，智能体有时候会在过程中迷失方向。

因此，我们选择替大语言模型执行代码。

我们采用传统的静态分析流水线：向 Python 代码中注入缓存注解，以避免重复执行；然后再通过一个定制框架运行它。

这里是一个基准测试示例，对比的是 Claude 自己调用代码执行，与口袋分析师的执行方式。

你们可以看到，我们再次更快，因为我们不会重复加载数据，也不会重复执行中间过程。

但真正的优势并不在第一次运行代码时，而在第二次运行时。

好了，这里展示的是一个基准测试：我们取一个计划中的最后一张图表，然后仅仅修改它的名称。

Claude Code 会重新运行所有代码，因此总体耗时基本相同，尽管编辑代码这一步确实会更快一些。

但口袋分析师在第二轮中，几乎能够实现瞬时的代码执行。

这意味着，当投资者使用这个产品时，他们可以对投资分析做一些细微修改，而不必承担常规迭代流程中反复加载和执行所带来的额外开销。

好的，我们时间到了。

我们有很多经验和收获，但遗憾的是，今天只能讲这些。

第一个核心结论是：我们非常相信让智能体专门化。

我们并不太相信通用的、强大的智能体。

它们确实能够做出非常酷的演示，我相信很多人都展示过这类演示；但要把它变成可以每天依赖的工作流程，真的非常困难。

相反，我们通常从非常狭窄的工作流程入手，对其进行大量基准测试；然后不断对这些基准进行爬坡式优化。

之后，你可以把多个智能体的能力组合起来；但反过来，从一个泛化智能体再回到可靠、狭窄的工作流，往往很难。

最后一点更像是一个思维实验，希望大家会觉得有启发：

应该把智能体编程视为一个编译器问题，而不是一个智能体问题。

编译器已经生成代码数十年了，并且积累了大量技术，告诉我们如何更可靠、更正确、更具确定性地生成代码。



所以，我们非常愿意继续和大家交流这些问题。

最后，我想感谢我们的直接团队成员。当然，还有很多其他人参与其中，但这是核心团队。

谢谢大家。

我们会在外面的 AMA（问答交流）环节，真诚希望能和大家聊天、讨论，看看你们是否也对这些问题感到兴奋。

也期待今晚在其他场合见到大家。

非常感谢。

本文来源：[AI原生Lab](#)

AI 原生 Lab

研究 AI 原生实践，重构组织与工作

拆解 AI 实践， 重构企业能力。

真实案例

方法提炼

落地复用



关注 AI 原生 Lab
获取更多 AI 原生实践

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码 免费领取

VIP会员 · 7天畅读卡 | 大师课 · 入门串讲



限免

48 收藏

分享到:

写评论

请发表您的评论

表情 图片

发布评论